

AniData Lab — Semaine 2

Git, GitHub & CI/CD

De la machine du data engineer à la production automatisée



Git



GitHub



CI/CD

Au programme aujourd'hui



Git

Versionner son code, travailler en branches, collaborer



GitHub

Héberger, partager, reviewer du code en équipe



CI/CD

Automatiser les tests et le déploiement avec GitHub Actions

Objectif fil rouge : un commit qui se propage automatiquement jusqu'au dashboard.

Pourquoi versionner son code ?



Historique complet

Retrouver qui a changé quoi, quand, et pourquoi. Revenir en arrière à n'importe quel moment.



Travailler à plusieurs

Plusieurs personnes éditent le même projet sans se marcher dessus.



Sauvegarde et confiance

Le code est répliqué localement et à distance. On essaie des choses sans peur de casser.

Sans Git, ça ressemble à ça...

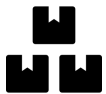
Sans Git

- rapport_v1.docx, rapport_v2.docx, rapport_FINAL_vraiFINAL.docx
- Une seule personne peut modifier à la fois
- Si on perd le fichier, on perd tout
- Aucune trace des modifications passées
- Les conflits se règlent par mail

Avec Git

- Une seule version vivante du projet
- Branches parallèles pour expérimenter
- Historique complet de chaque ligne
- Fusion structurée des changements
- Sauvegarde sur la machine + serveur distant

Quatre concepts à retenir



Repository

Le dossier de votre projet, suivi par Git.



Commit

Un instantané de votre projet à un moment donné, avec un message.



Branche

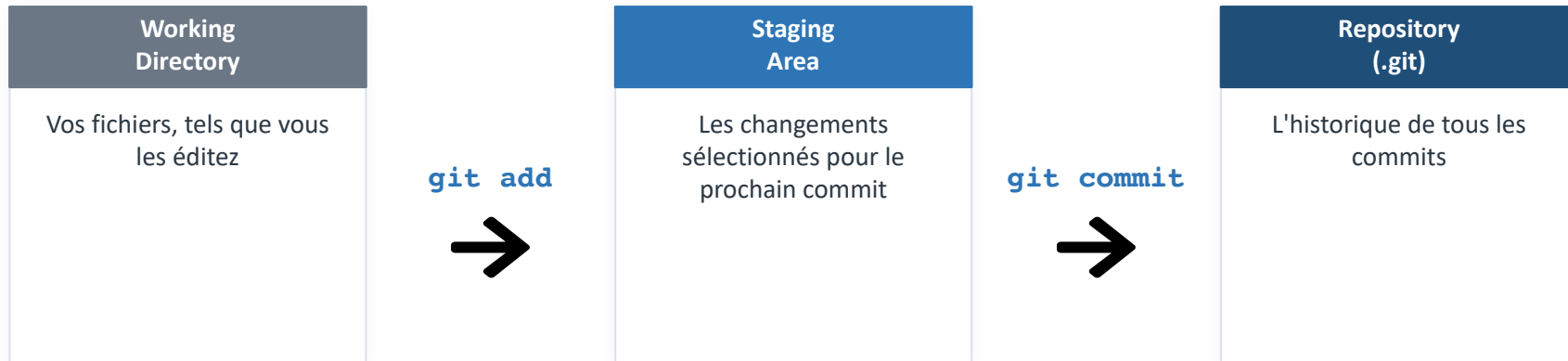
Une ligne de développement parallèle, indépendante du reste.



Remote

Une copie distante du repository (généralement sur GitHub).

Les trois zones de Git



`git checkout / git restore`
(récupérer une version antérieure depuis le repository)

Premiers pas : commandes essentielles

Le workflow de base, en 5 commandes :

```
$ git init                # Crée un nouveau repository
$ git status              # Voir l'état des fichiers
$ git add scraper.py      # Ajouter à la staging area
$ git commit -m "feat: ajout scraper" # Créer un commit
$ git log --oneline       # Voir l'historique
```



Astuce : un message de commit au présent, qui décrit l'intention. Pas "j'ai modifié".

Bien écrire ses messages de commit

À éviter

- fix
- modifs
- ça marche
- asdf
- merge

À privilégier

- feat: ajoute pagination du scraper
- fix: corrige parsing du score N/A
- docs: ajoute exemple Airflow
- refactor: extrait helper _fetch
- test: couvre la gestion des 5xx

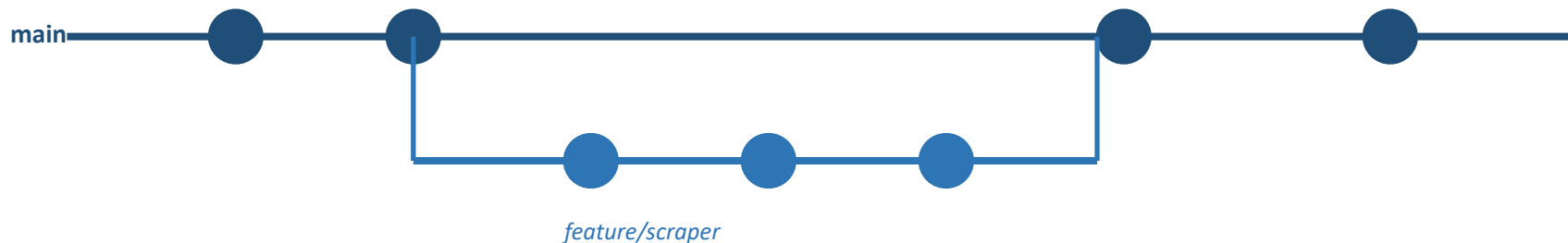
Format Conventional Commits

```
<type>(<scope facultatif>): <description courte à l'impératif>
```

```
feat, fix, docs, style, refactor, test, chore, ci
```


Travailler en branches

Une branche = une ligne de développement parallèle. Indispensable pour ne pas casser main.



- `git switch -c feature/scraper` crée et bascule sur une branche
- `git switch main` revient sur main
- `git merge feature/scraper` intègre les commits dans main

Workflow GitFlow allégé



On ne commit JAMAIS directement sur main : tout passe par une branche + une Pull Request.

Que faire d'un conflit de merge ?

Un conflit apparaît quand Git ne peut pas décider entre deux versions d'une même ligne.

```
<<<<<< HEAD
score = float(raw_score)
=====
score = float(raw_score) if raw_score != "N/A" else None
>>>>>> feature/handle-na
```

- Ouvrir le fichier, choisir la bonne version (ou combiner les deux)
- Supprimer les marqueurs <<<<<<, =====, >>>>>>
- `git add <fichier>` puis `git commit` pour finaliser le merge

.gitignore : ce qu'on ne versionne jamais

Tout ne mérite pas d'être commité. Le .gitignore liste ce que Git doit ignorer.

À ne jamais commiter

- Mots de passe, clés API, tokens
- Environnements virtuels (.venv/, node_modules/)
- Fichiers compilés (__pycache__/, *.pyc)
- Données volumineuses (data/, *.csv)
- Configurations locales (.env, .vscode/)

Exemple .gitignore Python

```
# Python
__pycache__/
*.pyc
.venv/

# Data
data/
*.json

# Secrets
.env
```

Git ≠ GitHub



Git

L'outil

- Logiciel installé sur votre machine
- Fonctionne en ligne de commande
- Versionne les fichiers localement
- Open source, créé en 2005



GitHub

La plateforme

- Service web qui héberge vos repositories
- Interface graphique riche
- Collaboration : Pull Requests, issues, reviews
- CI/CD intégrée (GitHub Actions)

Anatomie d'un repository GitHub

 github.com/sakura-analytics/anidata-scraper

Code	Issues	Pull Requests	Actions	Wiki	Settings
 <code>.github/workflows/</code>				<i>Workflows CI/CD</i>	
 <code>anidata_scraper/</code>				<i>Code Python</i>	
 <code>tests/</code>				<i>Tests unitaires</i>	
 <code>.gitignore</code>				<i>Fichiers à ignorer</i>	
 <code>Dockerfile</code>				<i>Image Docker</i>	
 <code>README.md</code>				<i>Documentation principale</i>	

Connecter son projet à GitHub

Deux flux possibles selon le point de départ :

Cas 1 — J'ai déjà du code en local

```
$ git remote add origin git@github.com:user/repo.git  
$ git branch -M main  
$ git push -u origin main
```

Cas 2 — Le repo existe déjà sur GitHub

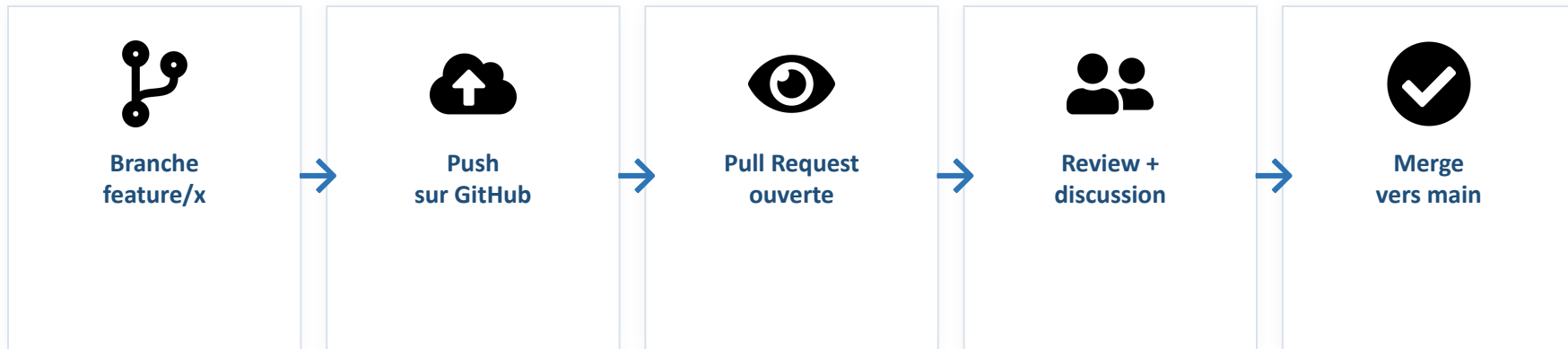
```
$ git clone git@github.com:user/repo.git
```



Recommandé : configurer une clé SSH pour ne pas saisir son mot de passe à chaque push.

La Pull Request, cœur de la collaboration

Une PR propose d'intégrer une branche dans une autre, avec discussion et review avant merge.



- Espace de discussion ligne par ligne sur le code proposé
- Validation explicite par un ou plusieurs binômiers ("Approve")
- Lien direct vers la CI : on ne merge que si les tests passent

À l'intérieur d'une Pull Request

#42 feat: ajoute la pagination du scraper



Description

Résumé du changement, contexte, lien vers l'issue traitée.



Checks (CI)

Lint et tests automatiques. Statut visible : ✓ / ✗



Reviewers

Personnes assignées pour relire et approuver le code.



Files changed

Diff ligne par ligne, avec commentaires en marge.



Merge

Bouton vert quand tout est validé. Sinon, on continue à pousser.

Bonnes pratiques de code review

Côté auteur de la PR

- PR petites (< 300 lignes idéalement)
- Description claire : pourquoi, pas seulement quoi
- Tests joints au changement
- Répondre aux commentaires, pas les ignorer

Côté reviewer

- Lire la description avant le code
- Critiquer le code, pas la personne
- Différencier bloquant / suggestion / nit
- Approuver dès que c'est bon, pas perfect

Issues : tracker le travail à faire

Une issue documente une tâche, un bug ou une idée. Centralise les discussions techniques.



Le scraper crash si le score vaut 'N/A'

bug

#37



Ajouter la pagination du catalogue

feature

#38



Documenter l'intégration au DAG Airflow

docs

#39

Une PR peut référencer une issue : « Closes #38 » → fermeture automatique au merge.

Protéger sa branche main

Sur GitHub, on configure des règles pour empêcher les erreurs sur main :



Pas de push direct

Tout passe par une Pull Request



Tests verts requis

La CI doit passer pour pouvoir merger



Au moins 1 review

Un binômier doit approuver la PR



Pas de force push

Garde l'historique propre et lisible

Le README, vitrine du projet

C'est la première chose que voit un visiteur. Un bon README répond à 4 questions :

Quoi ?

À quoi sert ce projet, en une phrase ?

Pourquoi ?

Quel problème résout-il ? Pour qui ?

Comment ?

Comment l'installer, le lancer, le tester ?

Et après ?

Comment contribuer ? Où trouver de l'aide ?

Récap GitHub : ce qu'on retient

- GitHub héberge le code, mais surtout la collaboration autour du code.
- Tout changement passe par une branche, puis une Pull Request.
- La PR permet la review, la discussion, et la validation par les pairs.
- Les issues centralisent les tâches, bugs et idées du projet.
- On protège main : pas de push direct, CI verte requise, review obligatoire.
- Un bon README est aussi important que du bon code.

CI/CD : automatiser la qualité et le déploiement

CI

Continuous Integration

À chaque commit, on vérifie automatiquement que le code compile, que les tests passent, que le style est respecté.

CD

Continuous Delivery / Deployment

Si la CI est verte, on construit automatiquement les artefacts (image Docker, binaires) et on les déploie.

Pourquoi automatiser ?



Détection précoce

Un bug attrapé en CI est 100x moins coûteux qu'en production.



Reproductibilité

Tout le monde teste avec le même environnement, pas de "ça marche chez moi".



Vitesse

Plus de validations manuelles longues. Push → résultat en 2 minutes.



Visibilité

Statut public sur chaque PR. Personne ne merge du rouge.

GitHub Actions : le vocabulaire



Workflow

Un ensemble de jobs qui s'exécutent en réponse à un événement (push, PR, tag...). Décrit dans un fichier YAML.



Job

Une unité d'exécution autonome, qui tourne sur un runner (machine virtuelle Linux/Mac/Windows fournie par GitHub).



Step

Une étape dans un job : commande shell, ou utilisation d'une action publique (ex: actions/checkout).



Action

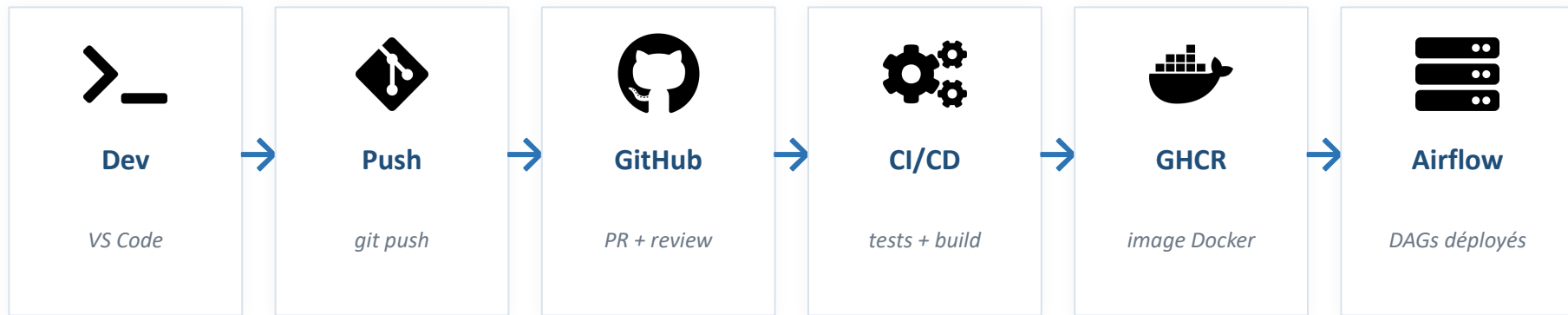
Un composant réutilisable, partagé sur le Marketplace. Évite de réécrire les briques courantes.

Mon premier workflow

Fichier : .github/workflows/ci.yml — déclenche lint + tests à chaque push et PR.

```
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.10' }
      - run: pip install -r requirements-dev.txt
      - run: ruff check .
      - run: pytest
```

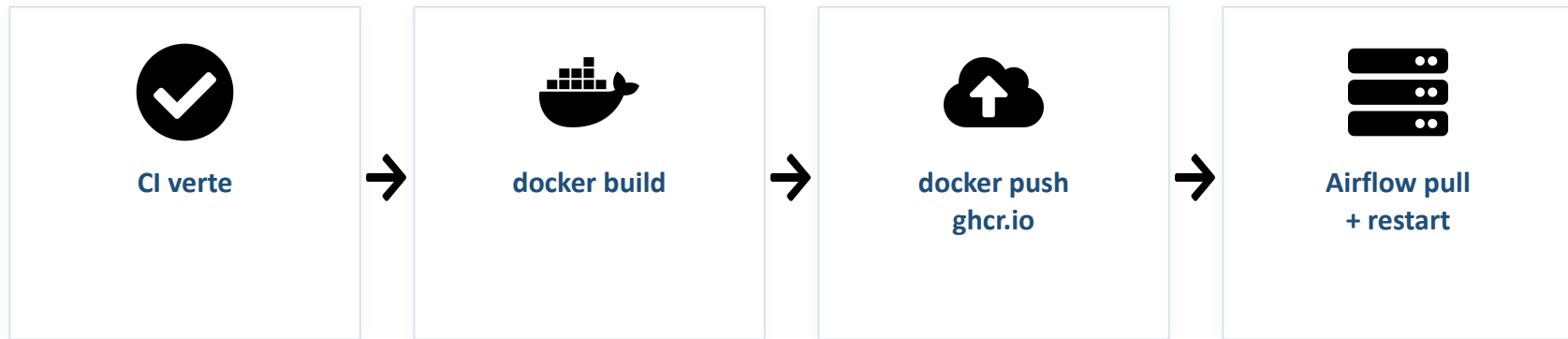
Notre pipeline AniData de bout en bout



Un seul commit déclenche toute la chaîne. C'est l'objectif de la semaine.

Le maillon CD : un registry d'images

Une fois la CI verte, on construit une image Docker contenant les DAGs et on la stocke quelque part.



GHCR (GitHub Container Registry) est gratuit, intégré à GitHub, et l'authentification se fait automatiquement avec le `GITHUB_TOKEN`.

Bonnes pratiques CI/CD



Tagger ses images

latest, SHA du commit, version sémantique.
Permet de revenir en arrière facilement.



Gérer les secrets

Jamais dans le code. On utilise les secrets
GitHub (settings → secrets).



Idempotence

Rejouer un pipeline doit donner le même
résultat. Pas d'effets de bord cachés.



Logs lisibles

Un échec doit pointer immédiatement vers la
cause. Logs = première ligne de debug.

À retenir pour la semaine



Git versionne, GitHub collabore, CI/CD automatise.



Une feature = une branche = une PR. On ne touche jamais main directement.



À chaque push : la CI tourne. À chaque merge : la CD déploie.



Notre objectif : un commit qui propage le scraping jusqu'au dashboard Grafana.

Place au TP — installons Git et créons notre premier repo.